Ancestors_TreeType/QueType.h

```
#ifndef QUETYPE_H
#define QUETYPE_H
// Header file for Queue ADT.
#include <new>
#include <cstddef>
class FullQueue
{};

class EmptyQueue
{};

template <class ItemType>
struct NodeType;

template <class ItemType>
class QueType
{
public:
    QueType();
    // Class constructor.
    // Because there is a default constructor, the precondition
    // that the queue has been initialized is omitted.
    QueType(int max);
    // Parameterized class constructor.
    ~QueType();
    // Class destructor.
    QueType(const QueType<ItemType>& anotherQue);
    // Copy constructor
    void MakeEmpty();
    // Function: Initializes the queue to an empty state.
    // Post: Queue is empty.
    bool IsEmpty() const;
    // Function: Determines whether the queue is empty.
    // Post: Function value = (queue is empty)
    bool IsFull() const;
    // Function: Determines whether the queue is full.
    // Post: Function value = (queue is full)
    void Enqueue(ItemType newItem);
    // Function: Adds newItem to the rear of the queue.
    // Post: If (queue is full) FullQueue exception is thrown
    //       else newItem is at rear of queue.
    void Dequeue(ItemType& item);
    // Function: Removes front item from the queue and returns it in item.
    // Post: If (queue is empty) EmptyQueue exception is thrown
    //       and item is undefined
    //       else front element has been removed from queue and
    //       item is a copy of removed element.
```

```
48  private:
49
50    NodeType<ItemType>* front;
51    NodeType<ItemType>* rear;
52  };
53
54  template <class ItemType>
55  struct NodeType
56  {
57    ItemType info;
58    NodeType* next;
59  };
60
61  template <class ItemType>
62  QueType<ItemType>::QueType()          // Class constructor.
63  // Post:  front and rear are set to NULL.
64  {
65    front = NULL;
66    rear = NULL;
67  }
68
69  template <class ItemType>
70  void QueType<ItemType>::MakeEmpty()
71  // Post: Queue is empty; all elements have been deallocated.
72  {
73    NodeType<ItemType>* tempPtr;
74
75      while (front != NULL)
76    {
77      tempPtr = front;
78      front = front->next;
79      delete tempPtr;
80    }
81    rear = NULL;
82  }
83
84  // Class destructor.
85  template <class ItemType>
86  QueType<ItemType>::~QueType()
87  {
88    MakeEmpty();
89  }
90
91  template <class ItemType>
92  bool QueType<ItemType>::IsFull() const
93  // Returns true if there is no room for another ItemType
94  //  on the free store; false otherwise.
95  {
96    NodeType<ItemType>* location;
```

```
  try
  {
    location = new NodeType<ItemType>;
    delete location;
    return false;
  }
  catch(std::bad_alloc)
  {
    return true;
  }
}

template <class ItemType>
bool QueType<ItemType>::IsEmpty() const
// Returns true if there are no elements on the queue; false otherwise.
{
  return (front == NULL);
}

template <class ItemType>
void QueType<ItemType>::Enqueue(ItemType newItem)
// Adds newItem to the rear of the queue.
// Pre:  Queue has been initialized.
// Post: If (queue is not full) newItem is at the rear of the queue;
//       otherwise a FullQueue exception is thrown.

{
  if (IsFull())
    throw FullQueue();
  else
  {
    NodeType<ItemType>* newNode;

    newNode = new NodeType<ItemType>;
    newNode->info = newItem;
    newNode->next = NULL;
    if (rear == NULL)
      front = newNode;
    else
      rear->next = newNode;
    rear = newNode;
  }
}

template <class ItemType>
void QueType<ItemType>::Dequeue(ItemType& item)
// Removes front item from the queue and returns it in item.
// Pre:  Queue has been initialized and is not empty.
// Post: If (queue is not empty) the front of the queue has been
```

```
146 //       removed and a copy returned in item;
147 //       othersiwe a EmptyQueue exception has been thrown.
148 {
149   if (IsEmpty())
150     throw EmptyQueue();
151   else
152   {
153     NodeType<ItemType>* tempPtr;
154
155     tempPtr = front;
156     item = front->info;
157     front = front->next;
158     if (front == NULL)
159       rear = NULL;
160     delete tempPtr;
161   }
162 }
163
164 template <class ItemType>
165 QueType<ItemType>::QueType(const QueType<ItemType>& anotherQue)
166 {
167   NodeType<ItemType>* ptr1;
168   NodeType<ItemType>* ptr2;
169
170   if (anotherQue.front == NULL)
171     front = NULL;
172   else
173   {
174     front = new NodeType<ItemType>;
175     front->info = anotherQue.front->info;
176     ptr1 = anotherQue.front->next;
177     ptr2 = front;
178     while (ptr1 != NULL)
179     {
180       ptr2->next = new NodeType<ItemType>;
181       ptr2 = ptr2->next;
182       ptr2->info = ptr1->info;
183       ptr1 = ptr1->next;
184     }
185     ptr2->next = NULL;
186     rear = ptr2;
187   }
188 }
189 #endif
190
```